

1. Education Analytics Dataset

```
import pandas as pd
import matplotlib.pyplot as plt
data = pd.read_csv(r"D:\4 SLOT\FODS\AT 3\student_performance.csv")
print("Student Performance Dataset")
print(data)

print("\nDataset Information")
print(data.info())
print("\nStatistical Summary")
print(data.describe())
print("\nMissing Values")
print(data.isnull().sum())
print("\nResult Count")
print(data["Result"].value_counts())
print("\nAverage Attendance:", data["Attendance"].mean())
print("Average Study Hours:", data["Study_Hours"].mean())
print("Average Internal Marks:", data["Internal_Marks"].mean())
plt.figure(figsize=(5,4))
plt.hist(data["Attendance"], bins=5)
plt.title("Attendance Distribution")
plt.xlabel("Attendance")
plt.ylabel("Number of Students")
plt.show()

plt.figure(figsize=(5,4))
plt.hist(data["Study_Hours"], bins=5)
plt.title("Study Hours Distribution")
plt.xlabel("Study Hours")
plt.ylabel("Number of Students")
plt.show()

plt.figure(figsize=(5,4))
plt.hist(data["Internal_Marks"], bins=5)
plt.title("Internal Marks Distribution")
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.show()

plt.figure(figsize=(5,4))
data["Result"].value_counts().plot(kind="bar")
plt.title("Student Result")
plt.xlabel("Result")
plt.ylabel("Count")
plt.show()
```

```
plt.figure(figsize=(5,4))
plt.scatter(data["Study_Hours"], data["Internal_Marks"])
plt.title("Study Hours vs Internal Marks")
plt.xlabel("Study Hours")
plt.ylabel("Internal Marks")
plt.show()

print("\nCorrelation Matrix")
print(data[["Attendance", "Study_Hours", "Internal_Marks"]].corr())
```

```
Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Student_ID      20 non-null    object
1   Attendance       20 non-null    int64
2   Study_Hours     20 non-null    int64
3   Internal_Marks  20 non-null    int64
4   Result          20 non-null    object
dtypes: int64(3), object(2)
memory usage: 932.0+ bytes
None
```

```
Statistical Summary
```

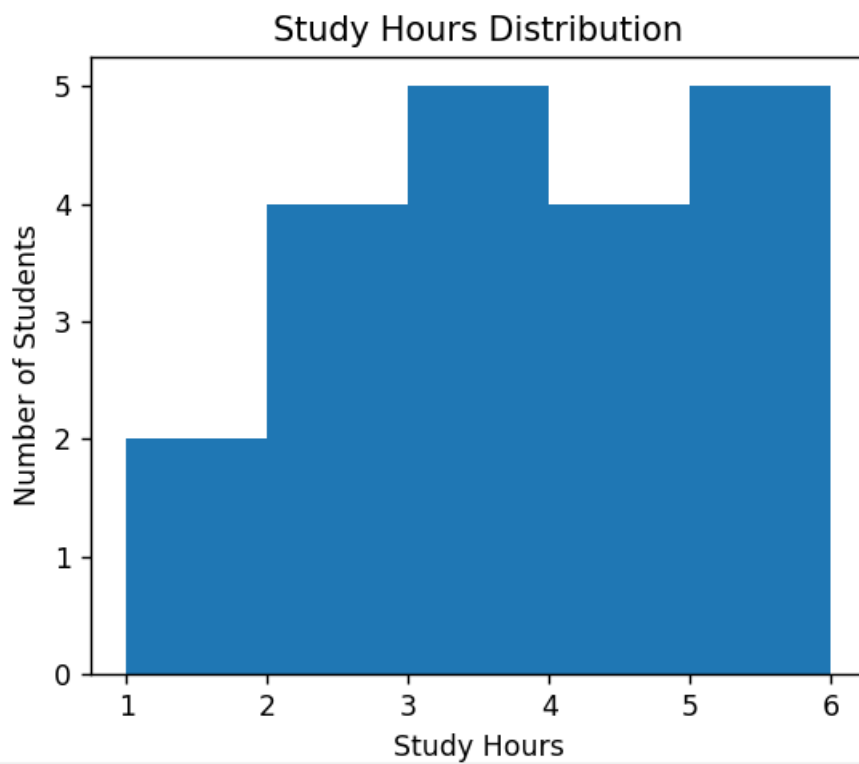
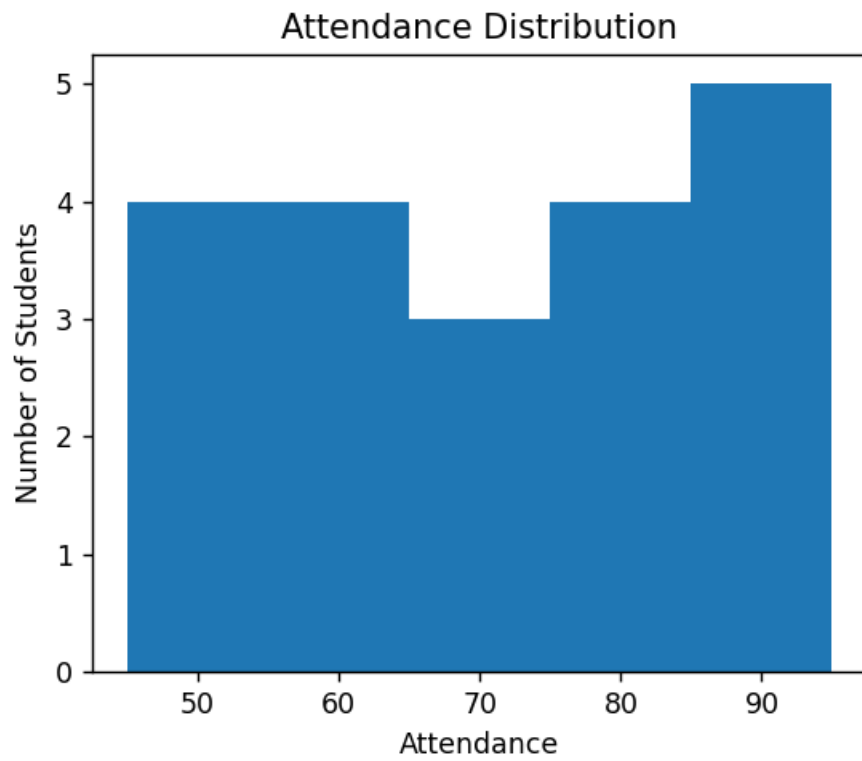
	Attendance	Study_Hours	Internal_Marks
count	20.000000	20.000000	20.000000
mean	70.150000	3.350000	32.950000
std	15.938286	1.424411	9.087151
min	45.000000	1.000000	18.000000
25%	57.250000	2.000000	24.750000
50%	70.000000	3.000000	33.500000
75%	83.500000	4.250000	40.250000
max	95.000000	6.000000	48.000000

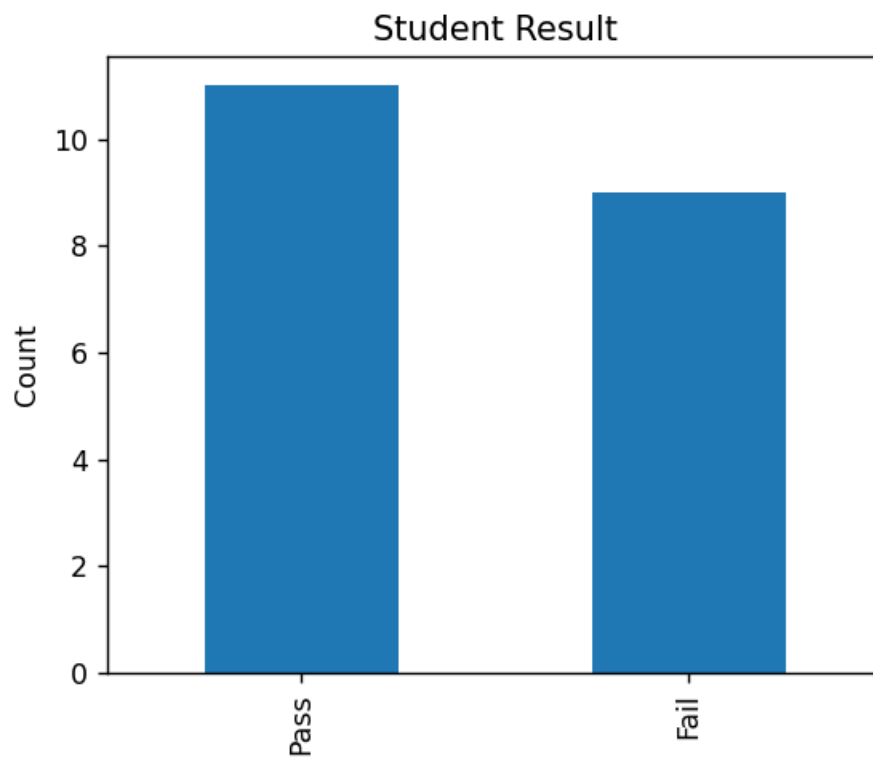
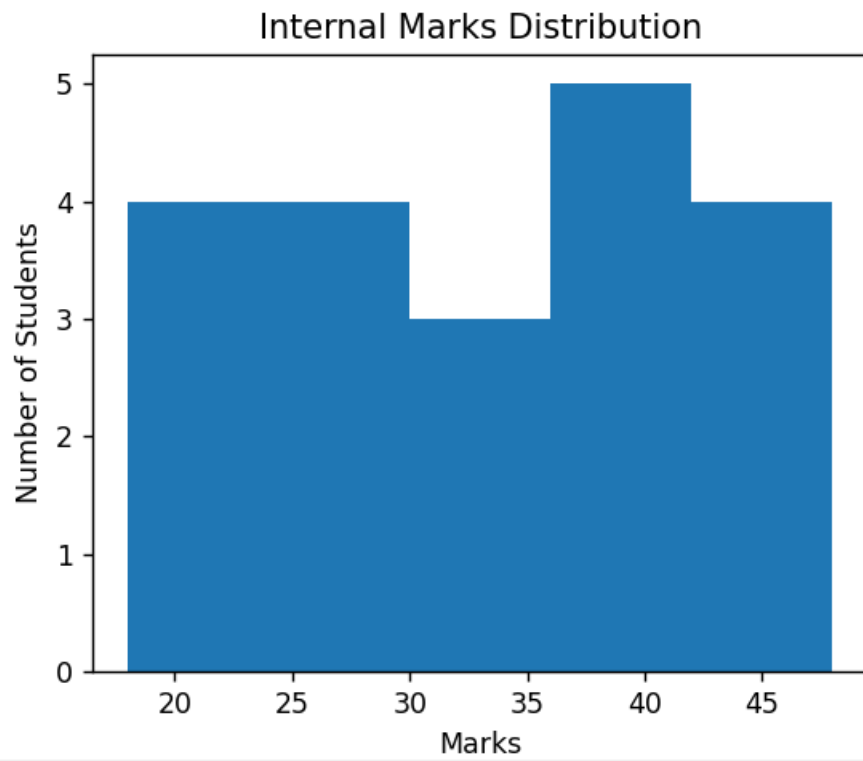
```
Missing Values
Student_ID      0
Attendance       0
Study_Hours     0
Internal_Marks  0
Result          0
dtype: int64
```

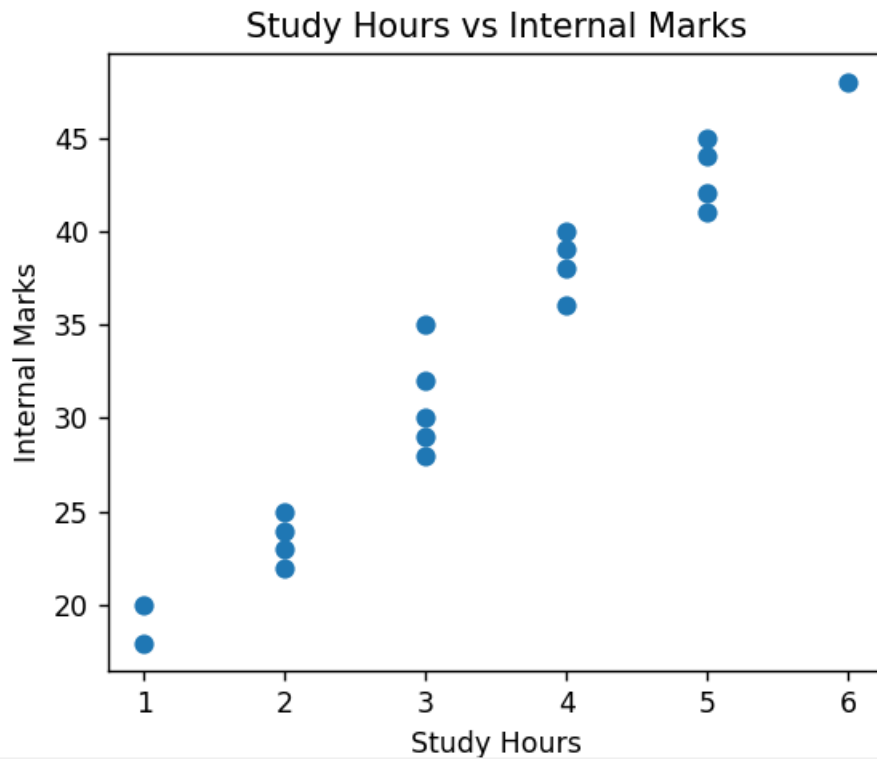
```
Result Count
Result
Pass    11
Fail     9
Name: count, dtype: int64
```

```
Average Attendance: 70.15
Average Study Hours: 3.35
Average Internal Marks: 32.95
```

```
|
```







2. Healthcare Analytics Dataset

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("D:/4 SLOT/FODS/AT 3/healthcare_analytics.csv")

print("Healthcare Analytics Dataset")
print(data)

print("\nRows and Columns")
print(data.shape)

print("\nData Types")
print(data.dtypes)

print("\nDataset Information")
print(data.info())

print("\nMissing Values")
print(data.isnull().sum())

print("\nDuplicate Values")
print(data.duplicated().sum())
```

```

print("\nStatistical Summary")
print(data.describe())

print("\nMean")
print(data[["Age", "Sugar_Level", "BP", "BMI"]].mean())

print("\nMedian")
print(data[["Age", "Sugar_Level", "BP", "BMI"]].median())

print("\nMinimum")
print(data[["Age", "Sugar_Level", "BP", "BMI"]].min())

print("\nMaximum")
print(data[["Age", "Sugar_Level", "BP", "BMI"]].max())

print("\nStandard Deviation")
print(data[["Age", "Sugar_Level", "BP", "BMI"]].std())

print("\nDisease Status Count")
print(data["Disease_Status"].value_counts())

print("\nCorrelation Matrix")
print(data[["Age", "Sugar_Level", "BP", "BMI"]].corr())

plt.figure(figsize=(5,4))
plt.hist(data["Sugar_Level"], bins=5)
plt.title("Sugar Level Distribution")
plt.xlabel("Sugar Level")
plt.ylabel("Patients")
plt.show()

plt.figure(figsize=(5,4))
plt.hist(data["BMI"], bins=5)
plt.title("BMI Distribution")
plt.xlabel("BMI")
plt.ylabel("Patients")
plt.show()

plt.figure(figsize=(5,4))
data["Disease_Status"].value_counts().plot(kind="bar")
plt.title("Disease Status")
plt.xlabel("Disease")
plt.ylabel("Count")
plt.show()

```

```
plt.figure(figsize=(5,4))
plt.scatter(data["Sugar_Level"], data["BMI"])
plt.title("Sugar Level vs BMI")
plt.xlabel("Sugar Level")
plt.ylabel("BMI")
plt.show()

plt.figure(figsize=(6,4))
data.boxplot(column=["Age","Sugar_Level","BP","BMI"])
plt.title("Box Plot")
plt.show()
```

OUTPUT:

Statistical Summary

	Age	Sugar_Level	BP	BMI
count	20.000000	20.000000	20.000000	20.000000
mean	43.200000	138.65000	129.750000	27.300000
std	11.316499	40.33449	14.523574	4.256265
min	27.000000	88.00000	108.000000	21.000000
25%	33.500000	99.50000	117.500000	23.750000
50%	43.500000	137.50000	129.000000	27.500000
75%	52.250000	171.25000	142.500000	30.250000
max	62.000000	210.00000	155.000000	35.000000

Mean

Age	43.20
Sugar_Level	138.65
BP	129.75
BMI	27.30

dtype: float64

Median

Age	43.5
Sugar_Level	137.5
BP	129.0
BMI	27.5

dtype: float64

Minimum

Age	27
Sugar_Level	88
BP	108
BMI	21

dtype: int64

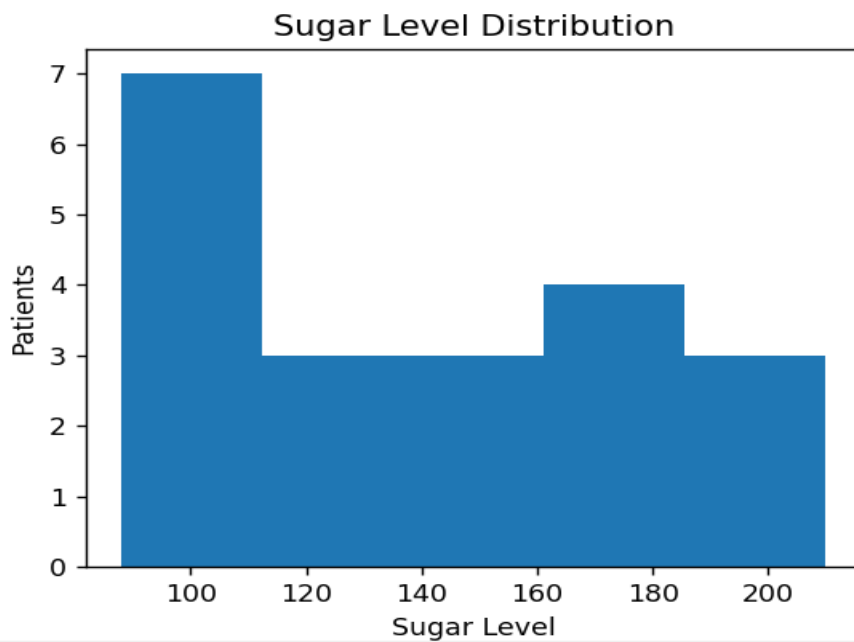
Maximum

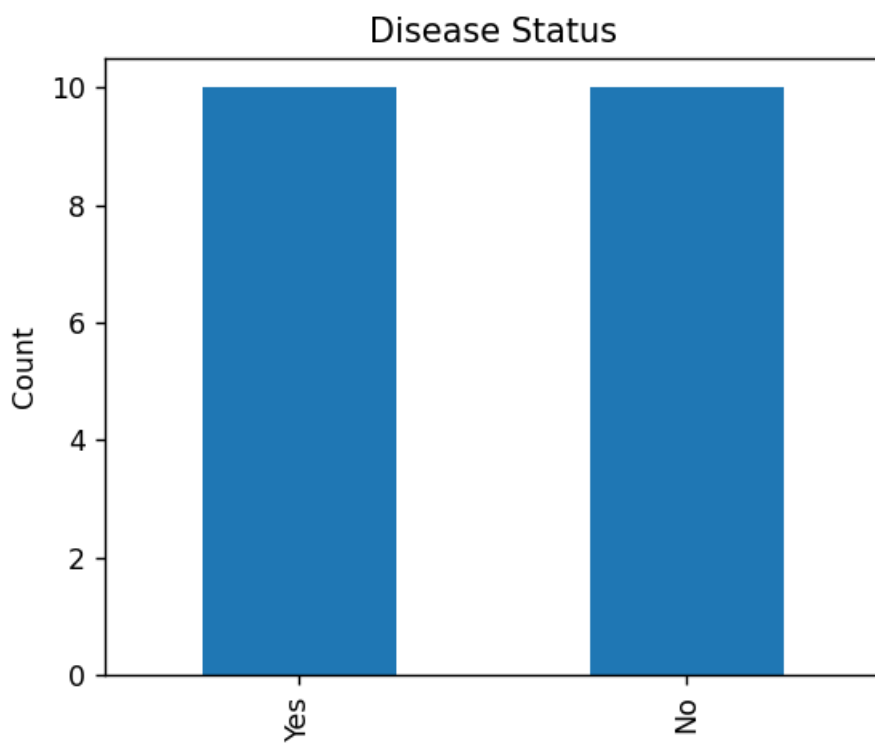
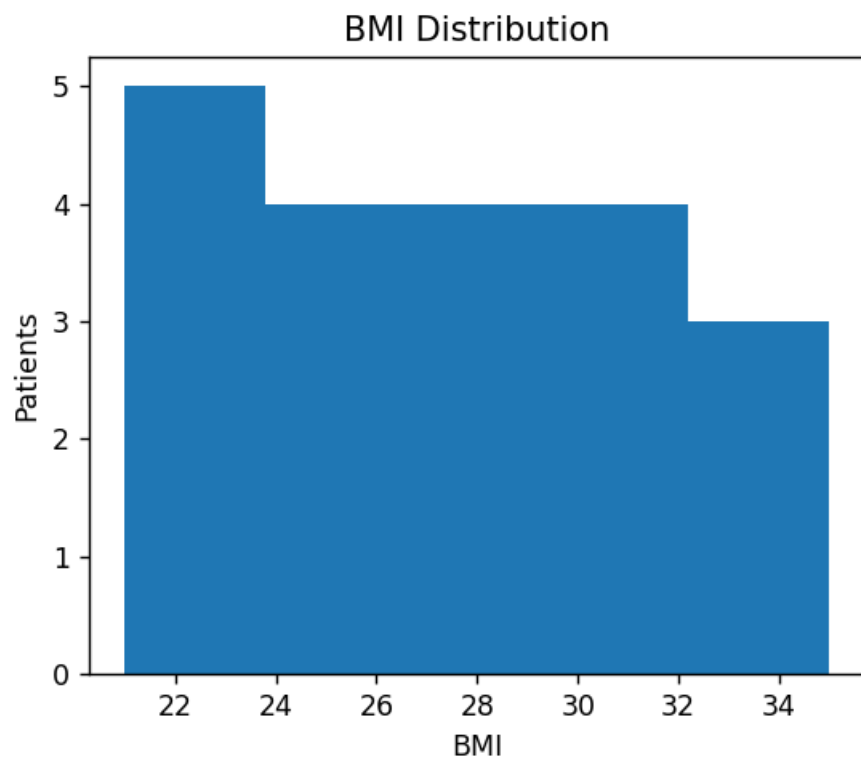
Age 62
Sugar_Level 210
BP 155
BMI 35
dtype: int64

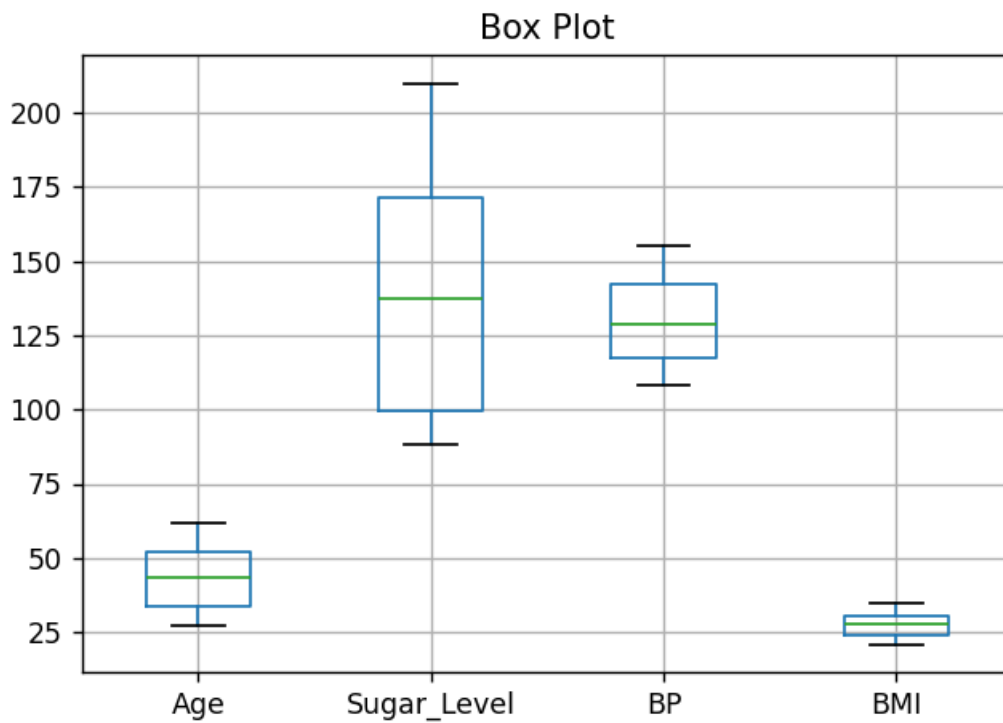
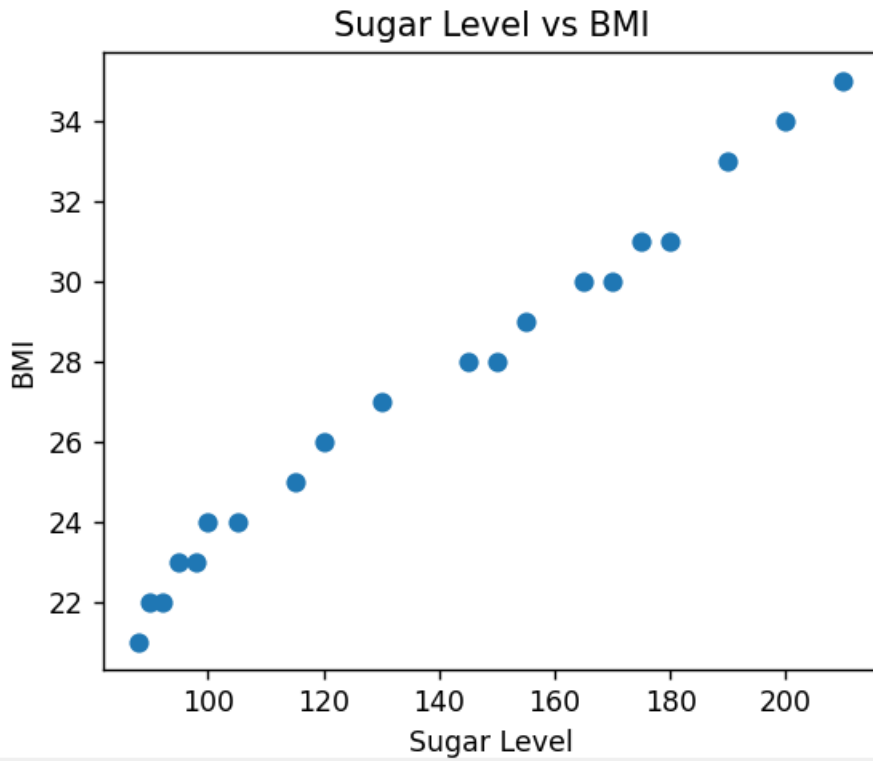
Standard Deviation
Age 11.316499
Sugar_Level 40.334490
BP 14.523574
BMI 4.256265
dtype: float64

Disease Status Count
Disease_Status
Yes 10
No 10
Name: count, dtype: int64

Correlation Matrix
Age Sugar_Level BP BMI
Age 1.000000 0.994112 0.994311 0.996335
Sugar_Level 0.994112 1.000000 0.993264 0.994264
BP 0.994311 0.993264 1.000000 0.993182
BMI 0.996335 0.994264 0.993182 1.000000







3. Retail Sales Dataset

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("D:/4 SLOT/FODS/AT 3/retail_sales.csv")

print("Retail Sales Dataset")
print(data)

print("\nRows and Columns")
print(data.shape)

print("\nData Types")
print(data.dtypes)

print("\nDataset Information")
print(data.info())

print("\nMissing Values")
print(data.isnull().sum())

print("\nDuplicate Values")
print(data.duplicated().sum())

print("\nStatistical Summary")
print(data.describe())

print("\nMean")
print(data[["Price", "Quantity_Sold", "Discount", "Revenue"]].mean())

print("\nMedian")
print(data[["Price", "Quantity_Sold", "Discount", "Revenue"]].median())

print("\nMinimum")
print(data[["Price", "Quantity_Sold", "Discount", "Revenue"]].min())

print("\nMaximum")
print(data[["Price", "Quantity_Sold", "Discount", "Revenue"]].max())

print("\nStandard Deviation")
print(data[["Price", "Quantity_Sold", "Discount", "Revenue"]].std())

print("\nCategory Count")
print(data["Category"].value_counts())
```

```

print("\nCorrelation Matrix")
print(data[["Price", "Quantity_Sold", "Discount", "Revenue"]].corr())

plt.figure(figsize=(5,4))
plt.hist(data["Revenue"], bins=5)
plt.title("Revenue Distribution")
plt.xlabel("Revenue")
plt.ylabel("Products")
plt.show()

plt.figure(figsize=(5,4))
plt.hist(data["Quantity_Sold"], bins=5)
plt.title("Quantity Sold Distribution")
plt.xlabel("Quantity Sold")
plt.ylabel("Products")
plt.show()

plt.figure(figsize=(5,4))
data["Category"].value_counts().plot(kind="bar")
plt.title("Product Categories")
plt.xlabel("Category")
plt.ylabel("Count")
plt.show()

plt.figure(figsize=(5,4))
plt.scatter(data["Price"], data["Revenue"])
plt.title("Price vs Revenue")
plt.xlabel("Price")
plt.ylabel("Revenue")
plt.show()

plt.figure(figsize=(6,4))
data.boxplot(column=["Price", "Quantity_Sold", "Discount", "Revenue"])
plt.title("Box Plot")
plt.show()

```

OUTPUT:

Statistical Summary

	Price	Quantity_Sold	Discount	Revenue
count	20.000000	20.000000	20.000000	20.000000
mean	54.000000	24.000000	4.000000	1114.000000
std	22.803509	9.673621	1.555973	221.416493
min	20.000000	10.000000	2.000000	810.000000
25%	35.000000	17.500000	3.000000	972.500000
50%	50.000000	21.000000	4.000000	1050.000000
75%	71.250000	30.500000	5.000000	1200.000000

max 100.000000 45.000000 7.000000 1540.000000

Mean

Price 54.0
Quantity_Sold 24.0
Discount 4.0
Revenue 1114.0
dtype: float64

Median

Price 50.0
Quantity_Sold 21.0
Discount 4.0
Revenue 1050.0
dtype: float64

Minimum

Price 20
Quantity_Sold 10
Discount 2
Revenue 810
dtype: int64

Maximum

Price 100
Quantity_Sold 45
Discount 7
Revenue 1540
dtype: int64

Standard Deviation

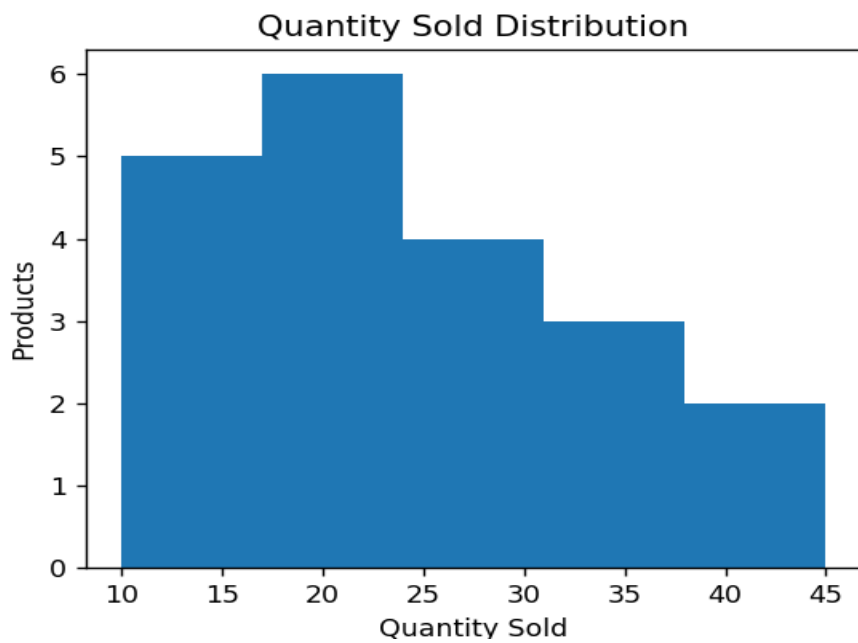
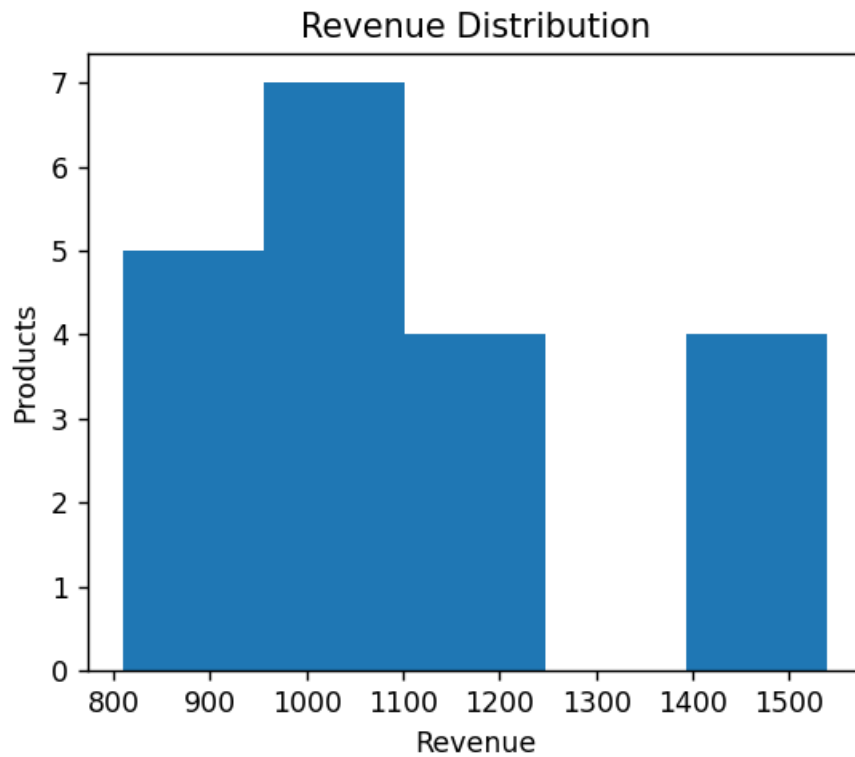
Price 22.803509
Quantity_Sold 9.673621
Discount 1.555973
Revenue 221.416493
dtype: float64

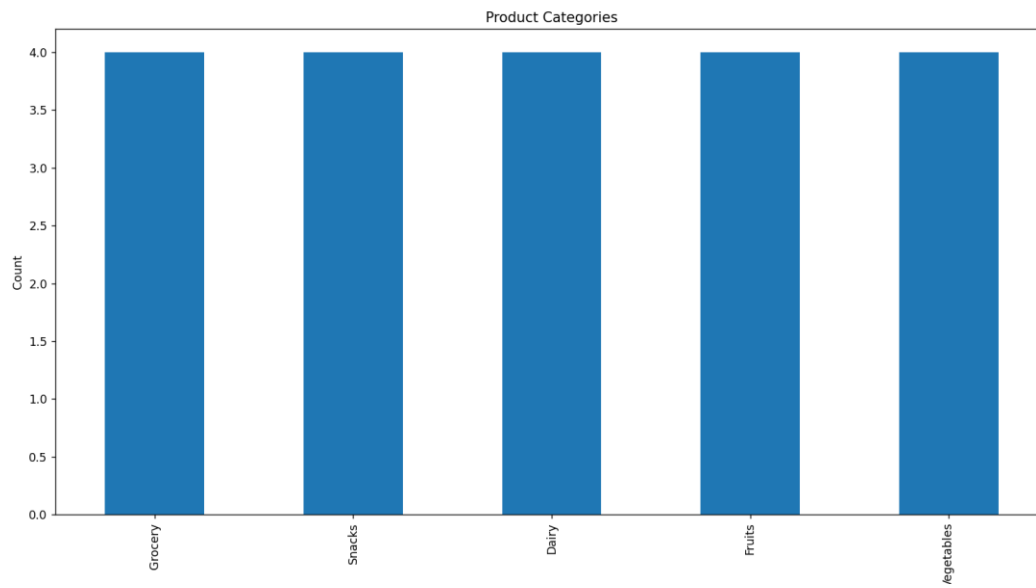
Category Count

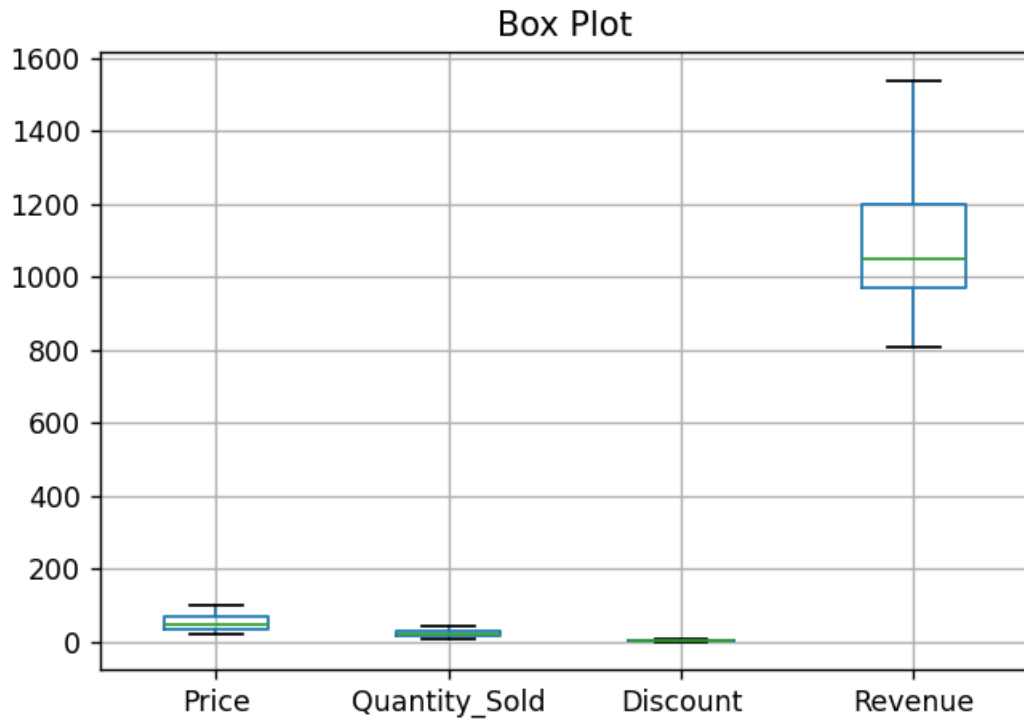
Category
Grocery 4
Snacks 4
Dairy 4
Fruits 4
Vegetables 4
Name: count, dtype: int64

Correlation Matrix

	Price	Quantity_Sold	Discount	Revenue
Price	1.000000	-0.868474	0.786174	0.388086
Quantity_Sold	-0.868474	1.000000	-0.507018	-0.081335
Discount	0.786174	-0.507018	1.000000	0.575938
Revenue	0.388086	-0.081335	0.575938	1.000000







4. Banking Loan Approval Dataset

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("D:/4 SLOT/FODS/AT 3/banking_loan.csv")

print("Banking Loan Approval Dataset")
print(data)

print("\nRows and Columns")
print(data.shape)

print("\nData Types")
print(data.dtypes)

print("\nDataset Information")
print(data.info())

print("\nMissing Values")
print(data.isnull().sum())

print("\nDuplicate Values")
print(data.duplicated().sum())
```



```
print("\nStatistical Summary")
print(data.describe())

print("\nMean")
print(data[["Income", "Credit_Score", "Loan_Amount"]].mean())

print("\nMedian")
print(data[["Income", "Credit_Score", "Loan_Amount"]].median())

print("\nMinimum")
print(data[["Income", "Credit_Score", "Loan_Amount"]].min())

print("\nMaximum")
print(data[["Income", "Credit_Score", "Loan_Amount"]].max())

print("\nStandard Deviation")
print(data[["Income", "Credit_Score", "Loan_Amount"]].std())

print("\nLoan Status Count")
print(data["Loan_Status"].value_counts())

print("\nCorrelation Matrix")
print(data[["Income", "Credit_Score", "Loan_Amount"]].corr())

plt.figure(figsize=(5,4))
plt.hist(data["Income"], bins=5)
plt.title("Income Distribution")
plt.xlabel("Income")
plt.ylabel("Customers")
plt.show()

plt.figure(figsize=(5,4))
plt.hist(data["Credit_Score"], bins=5)
plt.title("Credit Score Distribution")
plt.xlabel("Credit Score")
plt.ylabel("Customers")
plt.show()

plt.figure(figsize=(5,4))
data["Loan_Status"].value_counts().plot(kind="bar")
plt.title("Loan Approval Status")
plt.xlabel("Loan Status")
plt.ylabel("Count")
plt.show()

plt.figure(figsize=(5,4))
```

```
plt.scatter(data["Credit_Score"], data["Loan_Amount"])
plt.title("Credit Score vs Loan Amount")
plt.xlabel("Credit Score")
plt.ylabel("Loan Amount")
plt.show()

plt.figure(figsize=(6,4))
data.boxplot(column=["Income", "Credit_Score", "Loan_Amount"])
plt.title("Box Plot")
plt.show()
```

OUTPUT:

Statistical Summary

	Income	Credit_Score	Loan_Amount
count	20.000000	20.000000	20.000000
mean	43000.000000	664.000000	223250.000000
std	17923.815383	85.526235	83733.207272
min	22000.000000	555.000000	120000.000000
25%	26750.000000	583.750000	150000.000000
50%	40000.000000	660.000000	200000.000000
75%	56250.000000	735.000000	277500.000000
max	75000.000000	790.000000	400000.000000

Mean

Income	43000.0
Credit_Score	664.0
Loan_Amount	223250.0

dtype: float64

Median

Income	40000.0
Credit_Score	660.0
Loan_Amount	200000.0

dtype: float64

Minimum

Income	22000
Credit_Score	555
Loan_Amount	120000

dtype: int64

Maximum

Income	75000
Credit_Score	790
Loan_Amount	400000

dtype: int64

Standard Deviation

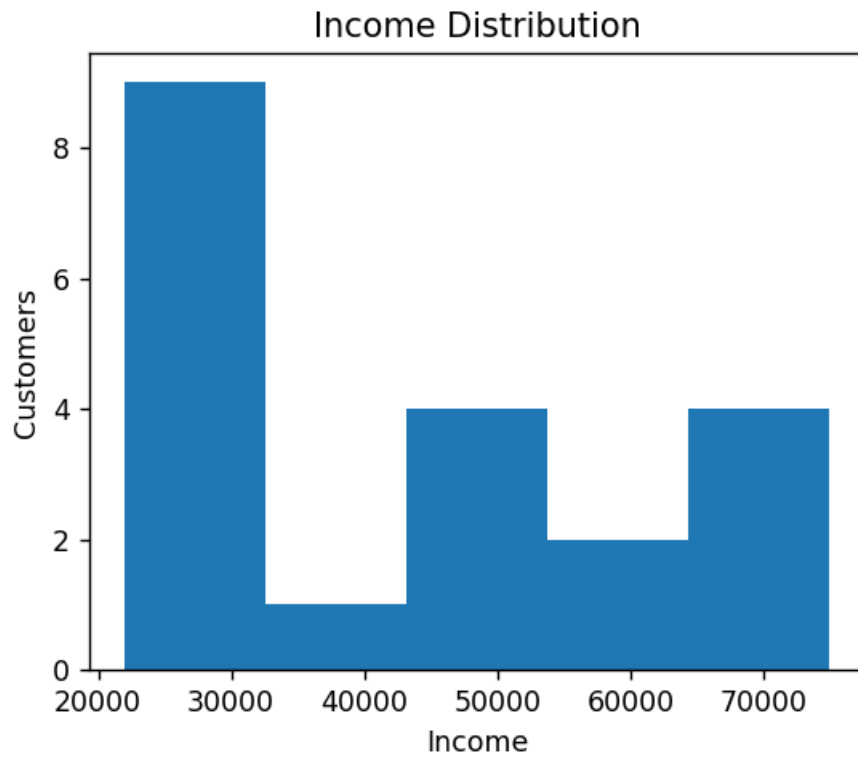
Income 17923.815383
Credit_Score 85.526235
Loan_Amount 83733.207272
dtype: float64

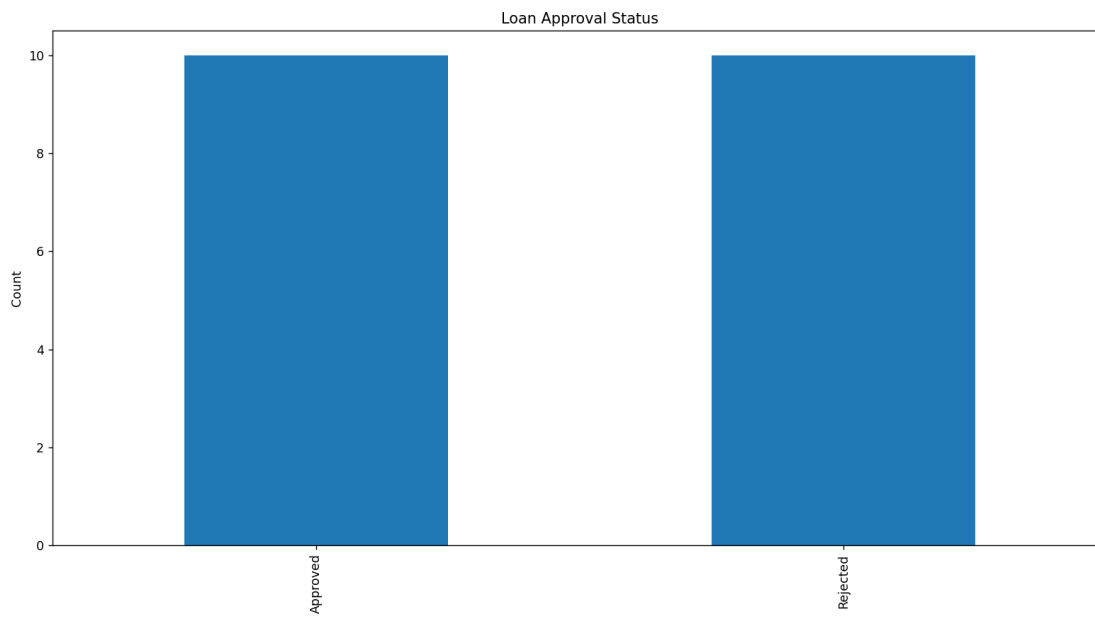
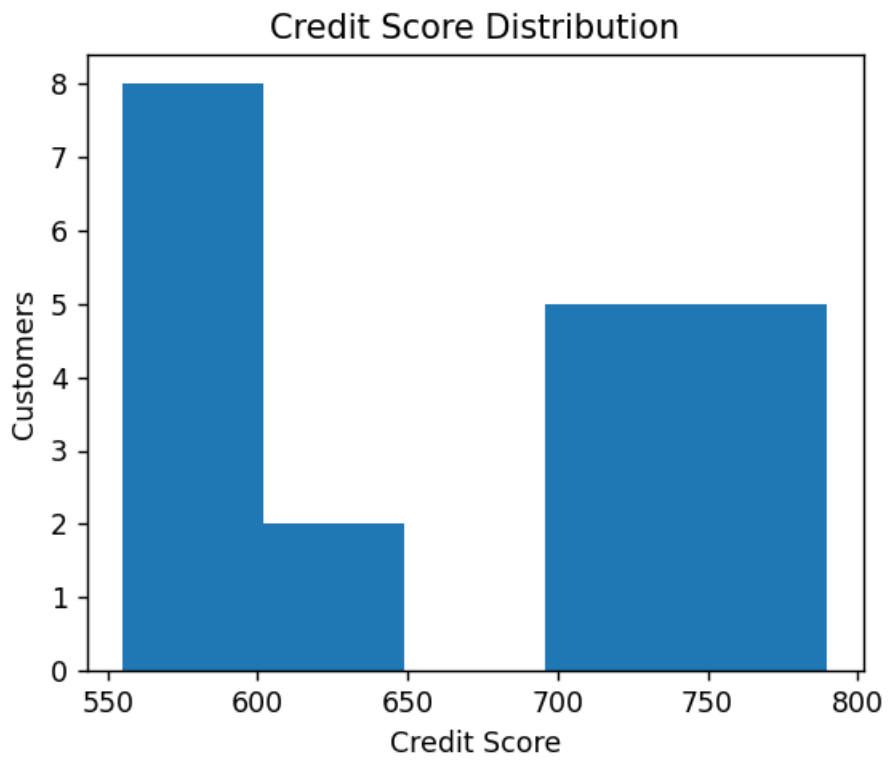
Loan Status Count

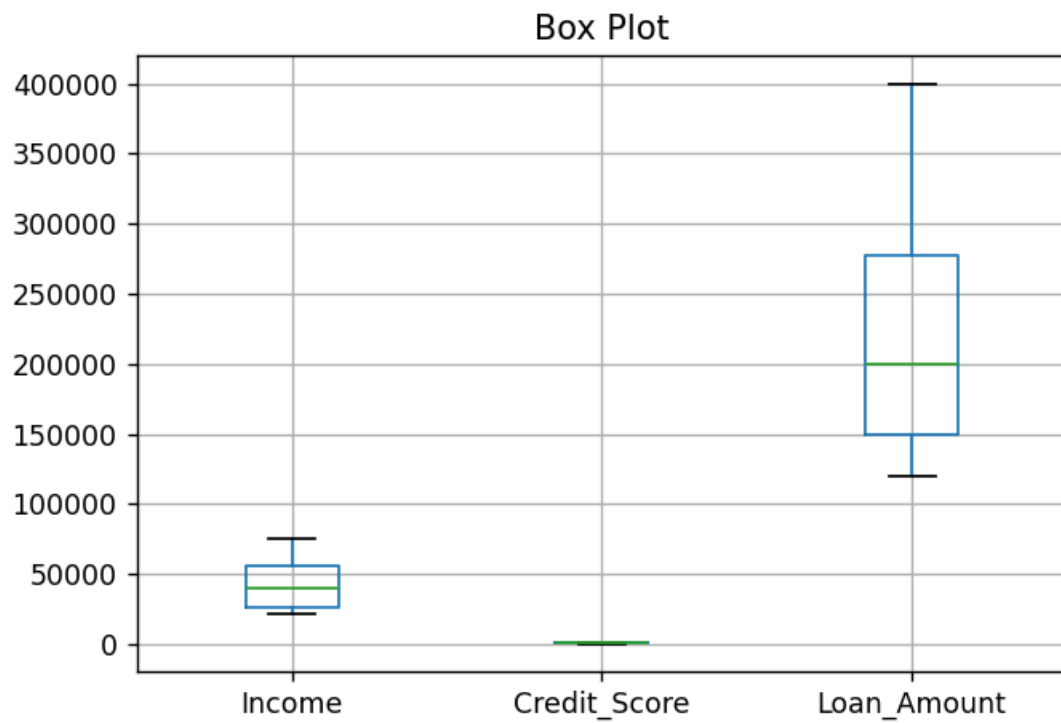
Loan_Status
Approved 10
Rejected 10
Name: count, dtype: int64

Correlation Matrix

	Income	Credit_Score	Loan_Amount
Income	1.000000	0.983995	0.987882
Credit_Score	0.983995	1.000000	0.953689
Loan_Amount	0.987882	0.953689	1.000000







5. Agriculture Crop Yield Dataset

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("D:/4 SLOT/FODS/AT 3/agriculture_crop_yield.csv")

print("Agriculture Crop Yield Dataset")
print(data)

print("\nRows and Columns")
print(data.shape)

print("\nData Types")
print(data.dtypes)

print("\nDataset Information")
print(data.info())

print("\nMissing Values")
print(data.isnull().sum())

print("\nDuplicate Values")
print(data.duplicated().sum())

print("\nStatistical Summary")
print(data.describe())

print("\nMean")
print(data[["Rainfall_mm", "Temperature", "Fertilizer_kg", "Crop_Yield_kg"]].mean())

print("\nMedian")
print(data[["Rainfall_mm", "Temperature", "Fertilizer_kg", "Crop_Yield_kg"]].median())

print("\nMinimum")
print(data[["Rainfall_mm", "Temperature", "Fertilizer_kg", "Crop_Yield_kg"]].min())

print("\nMaximum")
print(data[["Rainfall_mm", "Temperature", "Fertilizer_kg", "Crop_Yield_kg"]].max())

print("\nStandard Deviation")
print(data[["Rainfall_mm", "Temperature", "Fertilizer_kg", "Crop_Yield_kg"]].std())

print("\nSoil Type Count")
print(data["Soil_Type"].value_counts())
```

```

print("\nCorrelation Matrix")
print(data[["Rainfall_mm","Temperature","Fertilizer_kg","Crop_Yield_kg"]].corr())

plt.figure(figsize=(5,4))
plt.hist(data["Crop_Yield_kg"], bins=5)
plt.title("Crop Yield Distribution")
plt.xlabel("Crop Yield (kg)")
plt.ylabel("Farms")
plt.show()

plt.figure(figsize=(5,4))
plt.hist(data["Rainfall_mm"], bins=5)
plt.title("Rainfall Distribution")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Farms")
plt.show()

plt.figure(figsize=(5,4))
data["Soil_Type"].value_counts().plot(kind="bar")
plt.title("Soil Type")
plt.xlabel("Soil Type")
plt.ylabel("Count")
plt.show()

plt.figure(figsize=(5,4))
plt.scatter(data["Fertilizer_kg"], data["Crop_Yield_kg"])
plt.title("Fertilizer vs Crop Yield")
plt.xlabel("Fertilizer (kg)")
plt.ylabel("Crop Yield (kg)")
plt.show()

plt.figure(figsize=(6,4))
data.boxplot(column=["Rainfall_mm","Temperature","Fertilizer_kg","Crop_Yield_kg"])
plt.title("Box Plot")
plt.show()

```

OUTPUT:

Statistical Summary

	Rainfall_mm	Temperature	Fertilizer_kg	Crop_Yield_kg
count	20.000000	20.000000	20.000000	20.000000
mean	743.000000	29.550000	54.750000	2820.000000
std	144.335136	2.762055	10.507516	681.407213
min	500.000000	25.000000	40.000000	1800.000000
25%	615.000000	27.750000	45.750000	2175.000000
50%	770.000000	29.000000	54.500000	2975.000000
75%	857.500000	32.000000	60.500000	3250.000000

max 960.000000 34.000000 75.000000 3900.000000

Mean

Rainfall_mm 743.00
Temperature 29.55
Fertilizer_kg 54.75
Crop_Yield_kg 2820.00
dtype: float64

Median

Rainfall_mm 770.0
Temperature 29.0
Fertilizer_kg 54.5
Crop_Yield_kg 2975.0
dtype: float64

Minimum

Rainfall_mm 500
Temperature 25
Fertilizer_kg 40
Crop_Yield_kg 1800
dtype: int64

Maximum

Rainfall_mm 960
Temperature 34
Fertilizer_kg 75
Crop_Yield_kg 3900
dtype: int64

Standard Deviation

Rainfall_mm 144.335136
Temperature 2.762055
Fertilizer_kg 10.507516
Crop_Yield_kg 681.407213
dtype: float64

Soil Type Count

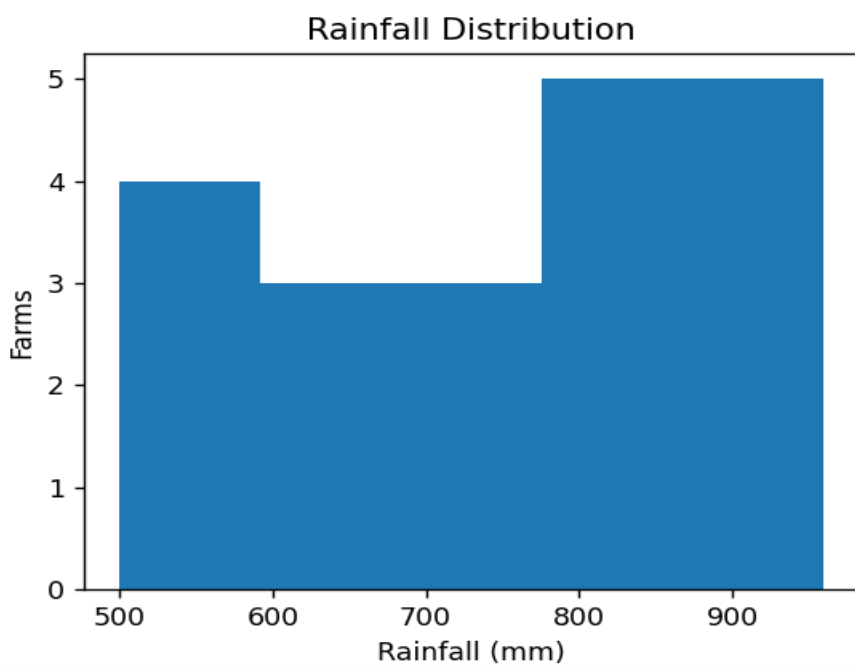
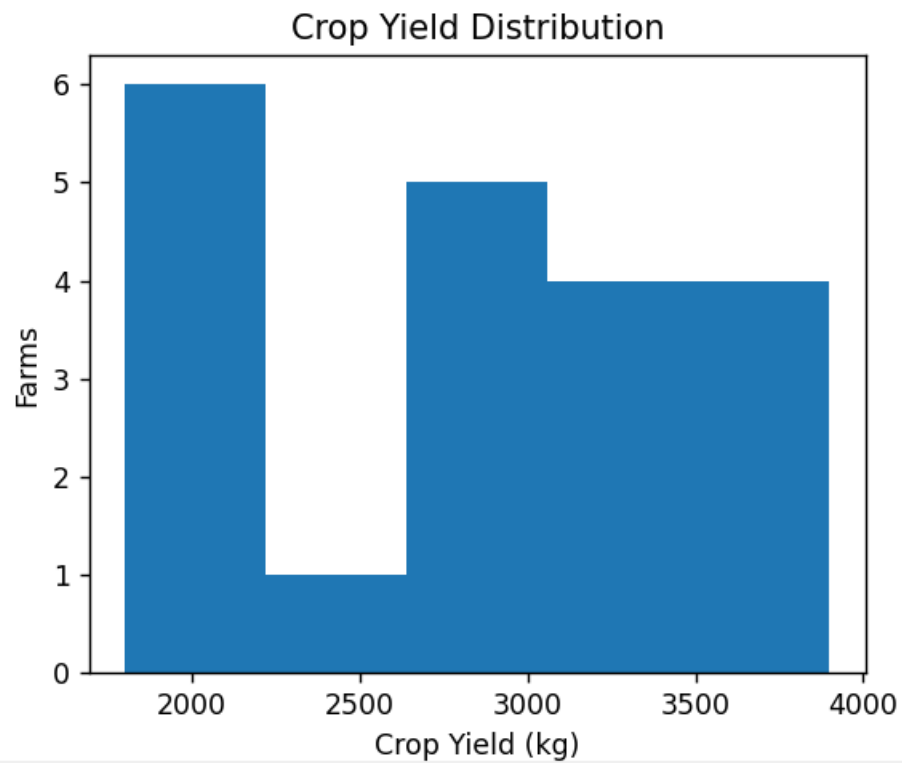
Soil_Type
Loamy 7
Sandy 7
Clay 6

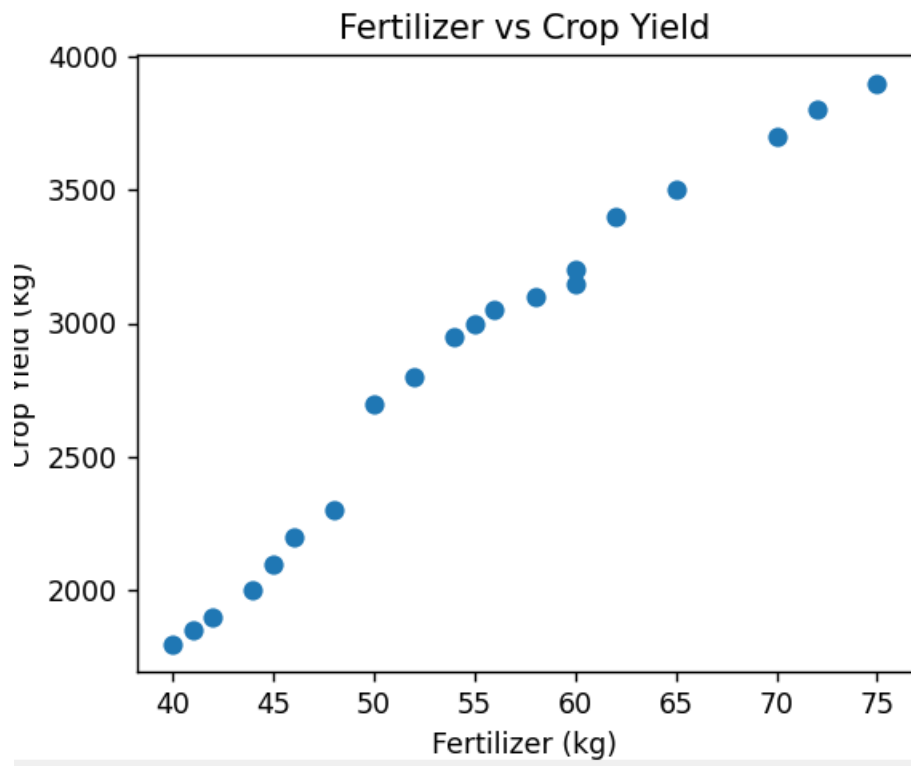
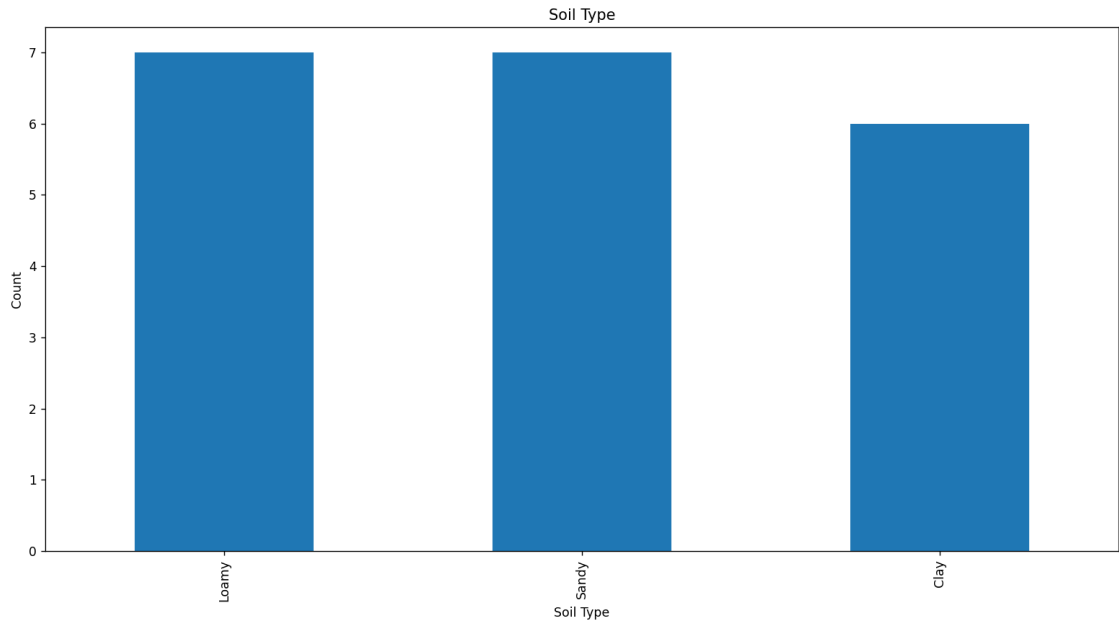
Name: count, dtype: int64

Correlation Matrix

Rainfall_mm Temperature Fertilizer_kg Crop_Yield_kg

Rainfall_mm	1.000000	-0.993192	0.980550	0.995521
Temperature	-0.993192	1.000000	-0.979735	-0.993300
Fertilizer_kg	0.980550	-0.979735	1.000000	0.983181
Crop_Yield_kg	0.995521	-0.993300	0.983181	1.000000





Box Plot

